لیست‌ها (Python Lists) در زبان برنامه‌نویسی پایتون

در بسیاری از برنامه‌ها لازم است چندین مقدار مرتبط را در یک متغیر ذخیره کنیم. برای مثال، ممکن است بخواهیم لیستی از نام دانشجویان، نمرات، شماره تلفن‌ها یا نام شهرها را نگهداری کنیم . در پایتون، **List** یکی از مهم‌ترین (Data Structures)است که برای ذخیره مجموعه‌ای از داده‌ها در یک متغیر استفاده می‌شود

لیست‌ها یکی از پرکاربردترین انواع داده در پایتون هستند و تقریباً در تمام برنامه‌های پایتون مورد استفاده قرار می‌گیرند

لیست(List) :

Listمجموعه‌ای از چندین مقدار (Item) است که در یک متغیر ذخیره می‌شوند لیست‌ها با استفاده از **(Square Brackets)** ایجاد می‌شوند

ساختار کلی:

```
list_name = [item1, item2, item3, ...]
```

مثال

```
fruits = ["Apple", "Banana", "Orange"]


print(fruits)
```

خروجی

```
['Apple', 'Banana', 'Orange']
```

چرا از List استفاده می‌کنیم؟

فرض کنید بخواهیم نام ۵ دانشجو را ذخیره کنیم

بدون:List

```
student1 = "Ali"
student2 = "Sara"
student3 = "Reza"
student4 = "Maryam"
```

```
student5 = "Nima"
```

با استفاده از List:

```
students = ["Ali", "Sara", "Reza", "Maryam", "Nima"]
```

همان اطلاعات تنها در یک متغیر ذخیره شده‌اند

ویژگی‌های لیست(List Items)

هر عضو موجود در لیست را **Item**می‌گویند

لیست‌ها دارای ویژگی‌های زیر هستند:

- Ordered ()
- Changeable ()
- Allow Duplicates ()

Ordered (مرتب بودن)

اعضای لیست دارای ترتیب مشخص هستند هر عضو دارای شماره‌ای به نام Indexاست اولین عضو دارای اندیس 0می‌باشد

مثال

```
fruits = ["Apple", "Banana", "Orange"]
print(fruits[0])
```

خروجی

Apple

```
print(fruits[1])
```

خروجی

Banana

```
print(fruits[2])
```

خروجی

```
Orange
```

افزودن عضو جدید :

اگر عضو جدیدی اضافه شود، در انتهای لیست قرار می‌گیرد

```
fruits = ["Apple", "Banana"]
fruits.append("Orange")
print(fruits)
```

خروجی

```
['Apple', 'Banana', 'Orange']
```

Changeable قابل تغییر بودن

برخلاف رشته‌ها(Strings) ، لیست‌ها قابل تغییر هستند یعنی می‌توان بعد از ایجاد لیست، اعضای آن را تغییر داد

مثال

```
fruits = ["Apple", "Banana", "Orange"]
fruits[1] = "Mango"
print(fruits)
```

خروجی

```
['Apple', 'Mango', 'Orange']
```

همچنین می‌توان عضو جدید اضافه یا عضوهای قبلی را حذف کرد

Allow Duplicates اعضای تکراری

لیست‌ها اجازه می‌دهند چند عضو مقدار یکسان داشته باشند

مثال

```
numbers = [10, 20, 20, 30, 20]

print(numbers)
```

خروجی

```
[10, 20, 20, 30, 20]
```

در این مثال، عدد 20سه بار در لیست وجود دارد

اندیس‌گذاری(Indexing)

اعضای لیست از صفر شماره‌گذاری می‌شوند

مقدار	اندیس
Apple	0
Banana	1
Orange	2
Mango	3

مثال

```
fruits = ["Apple", "Banana", "Orange", "Mango"]

print(fruits[3])
```

خروجی

```
Mango
```

اندیس منفی

در پایتون می‌توان از انتهای لیست نیز به اعضا دسترسی داشت

مقدار	اندیس
Mango	-1
Orange	-2
Banana	-3
Apple	-4

مثال

```
print(fruits[-1])
```

خروجی

Mango

طول لیست(List Length)

برای به دست آوردن تعداد اعضای لیست از تابع ()lenاستفاده می‌کنیم.

ساختار

```
len(list_name)
```

مثال

```
fruits = ["Apple", "Banana", "Orange"]
print(len(fruits))
```

خروجی

3

انواع داده در لیست(List Items - Data Types)

اعضای لیست می‌توانند از هر نوع داده‌ای باشند

لیست رشته‌ها

```
names = ["Ali", "Sara", "Maryam"]
```

لیست اعداد صحیح

```
numbers = [10, 20, 30]
```

لیست اعداد اعشاری

```
prices = [10.5, 25.8, 99.9]
```

لیست مقادیر Boolean

```
status = [True, False, True]
```

لیست ترکیبی

در پایتون اعضای لیست می‌توانند از انواع مختلف باشند.

```
person = ["Ali", 20, 18.5, True]
```

بررسی نوع داده(type)

برای بررسی نوع یک متغیر از تابع type()استفاده می‌کنیم

```
fruits = ["Apple", "Banana"]
print(type(fruits))
```

خروجی

```
<class 'list'>
```

از دید پایتون، لیست‌ها یک شیء (Object) از نوع listهستند

سازنده()list

علاوه بر استفاده از براکت‌ها، می‌توان با استفاده از تابع ()listنیز لیست ایجاد کرد

مثال

```
fruits = list(("Apple", "Banana", "Orange"))
print(fruits)
```

خروجی

```
['Apple', 'Banana', 'Orange']
```

توجه :هنگام استفاده از ()listباید مقادیر داخل یک تاپل یا یک مجموعه قابل پیمایش (Iterable) قرار گیرند

چهار نوع اصلی Collection در پایتون :

پایتون چهار نوع اصلی برای ذخیره مجموعه‌ای از داده‌ها دارد

نوع داده	مرتب	قابل تغییر	اعضای تکراری
List	بله	بله	بله
Tuple	بله	خیر	بله
Set	خیر	خیر*	خیر
Dictionary	بله) از Python 3.7 به بعد(	بله	کلیدهای تکراری ندارد

*خود مجموعه (Set) قابل تغییر است و می‌توان اعضایی را به آن اضافه یا حذف کرد، اما هر عضو باید یکتا باشد و ترتیب مشخصی ندارد

مقایسه List با سایر Collection ها

List

- دارای ترتیب مشخص
- قابل تغییر
- امکان وجود اعضای تکراری
- دسترسی با اندیس

```
colors = ["Red", "Blue", "Green"]
```

Tuple

- دارای ترتیب
- غیرقابل تغییر (Immutable)
- اعضای تکراری مجاز هستند .

numbers = (10, 20, 30)

Set

- بدون ترتیب
- بدون اندیس
- اعضای تکراری مجاز نیستند .

fruits = {"Apple", "Banana", "Orange"}

Dictionary :

- داده‌ها به صورت کلید (Key) و مقدار (Value) ذخیره می‌شوند
- کلیدها تکراری نیستند

```
student = {
   "name": "Ali",
   "age": 20
}
```

مثال کامل

```
students = ["Ali", "Sara", "Reza"]
print(students)
print(len(students))
print(students[0])
students[1] = "Maryam"
students.append("Nima")
print(students)
print(type(students))
```

خروجی

```
['Ali', 'Sara', 'Reza']
3
Ali
['Ali', 'Maryam', 'Reza', 'Nima']
<class 'list'>
```

کاربردهای List :

لیست‌ها در بسیاری از برنامه‌ها استفاده می‌شوند، از جمله:

- ذخیره نام دانشجویان
- ذخیره نمرات
- نگهداری شماره تلفن‌ها
- فهرست محصولات فروشگاه
- لیست فایل‌ها

- ذخیره پیام‌ها
- نتایج جستجو
- مدیریت کاربران

نکات مهم

- لیست‌ها با []ساخته می‌شوند.
- اعضای لیست دارای اندیس (Index) هستند و شماره‌گذاری از 0شروع می‌شود
- لیست‌ها مرتب (Ordered) هستند
- لیست‌ها قابل تغییر (Changeable) هستند
- اعضای تکراری در لیست مجاز هستند
- اعضای لیست می‌توانند از هر نوع داده‌ای باشند
- با تابع ()lenتعداد اعضای لیست به دست می‌آید
- با تابع ()typeمی‌توان نوع داده متغیر را بررسی کرد
- علاوه بر براکت‌ها، می‌توان از سازنده ()listبرای ایجاد لیست استفاده کرد

دسترسی به اعضای لیست (Access List Items) و تغییر آن‌ها در پایتون :

یکی از مهم‌ترین قابلیت‌های Listدر پایتون، دسترسی آسان به اعضا، تغییر مقدار آن‌ها و اضافه کردن عناصر جدید است. از آنجایی که لیست‌ها قابل تغییر (Mutable) هستند، می‌توان پس از ایجاد لیست، اعضای آن را ویرایش، حذف یا اضافه کرد

در این قسمت از درس با موارد زیر آشنا می‌شویم:

- دسترسی به اعضای لیست
- اندیس‌گذاری مثبت و منفی
- دسترسی به بازه‌ای از اعضا (Slicing)
- تغییر یک عضو
- تغییر چند عضو

- اضافه کردن عضو با insert()

دسترسی به اعضای لیست(Access Items) :

هر عضو در یک لیست دارای یک اندیس (Index) است که از 0شروع می‌شود

ساختار کلی:

list_name[index]

مثال

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
```

خروجی

apple

دسترسی به عضو دوم

```
print(fruits[1])
```

خروجی

banana

دسترسی به عضو سوم

```
print(fruits[2])
```

خروجی

cherry

اندیس‌های لیست :

اندیس	مقدار
0	apple
1	banana
2	cherry

اندیس منفی(Negative Indexing) :

در پایتون می‌توان از انتهای لیست نیز به اعضا دسترسی داشت

در اندیس‌های منفی:

- آخرین عضو -1
- عضو قبل از آخر -2
- سومین عضو از انتها -3

مثال

```
fruits = ["apple", "banana", "cherry"]
print(fruits[-1])
```

خروجی

cherry

```
print(fruits[-2])
```

خروجی

banana

```
print(fruits[-3])
```

خروجی

apple

جدول اندیس‌های مثبت و منفی :

مقدار	اندیس منفی	اندیس مثبت
apple	-3	0
banana	-2	1
cherry	-1	2

دسترسی به بازه‌ای از اعضا(Range of Indexes) :

می‌توان چند عضو متوالی را به صورت یک لیست جدید دریافت کرد

ساختار:

list[start:end]

در این حالت:

- startشامل نتیجه می‌شود
- endشامل نتیجه نمی‌شود

مثال

```
fruits = ["apple", "banana", "cherry", "orange", "kiwi"]

print(fruits[1:4])
```

خروجی

['banana', 'cherry', 'orange']

از ابتدای لیست :

اگر مقدار شروع را ننویسیم، پایتون از اندیس صفر شروع می‌کند

```
print(fruits[:3])
```

خروجی

```
['apple', 'banana', 'cherry']
```

تا انتهای لیست

اگر مقدار پایان را ننویسیم، پایتون تا آخر لیست ادامه می‌دهد

```
print(fruits[2:])
```

خروجی

```
['cherry', 'orange', 'kiwi']
```

استفاده از اندیس منفی درSlicing :

```
print(fruits[-4:-1])
```

خروجی

```
['banana', 'cherry', 'orange']
```

تغییر مقدار یک عضو(Change Item Value) :

برای تغییر مقدار یک عضو، کافی است از اندیس آن استفاده کنیم

ساختار:

```
list[index] = value
```

مثال

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "orange"
print(fruits)
```

خروجی

```
['apple', 'orange', 'cherry']
```

تغییر چند عضو(Change a Range of Item Values) :

می‌توان چند عضو را به صورت هم‌زمان تغییر داد

ساختار:

```
list[start:end] = [new values]
```

مثال

```
fruits = ["apple", "banana", "cherry", "orange"]
fruits[1:3] = ["blackcurrant", "watermelon"]
print(fruits)
```

خروجی

```
['apple', 'blackcurrant', 'watermelon', 'orange']
```

در این مثال:

- banana
- cherry

با دو مقدار جدید جایگزین شده‌اند

جایگزینی با تعداد بیشتری از اعضا :

اگر تعداد اعضای جدید بیشتر از اعضای قبلی باشد، طول لیست افزایش پیدا می‌کند

مثال

```
thislist = ["apple", "banana", "cherry"]
thislist[1:2] = ["blackcurrant", "watermelon"]
print(thislist)
```

خروجی

```
['apple', 'blackcurrant', 'watermelon', 'cherry']
```

توضیح:

در اینجا فقط عضو banana حذف شده است، اما به جای آن دو عضو جدید قرار گرفته‌اند؛ بنابراین طول لیست از ۳ به ۴ افزایش یافته است

جایگزینی با تعداد کمتر :

اگر تعداد اعضای جدید کمتر از اعضای قبلی باشد، طول لیست کاهش پیدا می‌کند

مثال

```
fruits = ["apple", "banana", "cherry", "orange"]
fruits[1:3] = ["kiwi"]
print(fruits)
```

خروجی

```
['apple', 'kiwi', 'orange']
```

در این مثال:

- دو عضو حذف شده‌اند
- فقط یک عضو جایگزین شده است

بنابراین طول لیست کمتر شده است

نکته مهم

اگر تعداد اعضای جایگزین با تعداد اعضای حذف‌شده برابر نباشد، طول لیست تغییر می‌کند

درج عضو جدید(Insert Items) :

گاهی نمی‌خواهیم عضوی را جایگزین کنیم، بلکه می‌خواهیم یک عضو جدید را در محل مشخصی وارد کنیم برای این کار از متد insert()استفاده می‌کنیم.

ساختار:

list.insert(index, value)

مثال

```
fruits = ["apple", "banana", "cherry"]
fruits.insert(2, "watermelon")
print(fruits)
```

خروجی

```
['apple', 'banana', 'watermelon', 'cherry']
```

در این مثال:

- عضو جدید در اندیس **2**قرار گرفته است
- سایر اعضا یک خانه به سمت راست منتقل شده‌اند

مثال دیگر

```
numbers = [10, 20, 40]
numbers.insert(2, 30)
print(numbers)
```

خروجی

```
[10, 20, 30, 40]
```

تفاوت insert()و تغییر مقدار :

اگر بنویسیم:

```
numbers[2] = 30
```

مقدار قبلی جایگزین می‌شود

اما اگر بنویسیم:

```
numbers.insert(2, 30)
```

عضو جدید اضافه می‌شود و مقدار قبلی حذف نمی‌شود

مثال کامل

```
fruits = ["apple", "banana", "cherry", "orange"]

print(fruits[0])

print(fruits[-1])

print(fruits[1:3])

fruits[2] = "kiwi"

print(fruits)

fruits[1:3] = ["grape", "melon"]

print(fruits)

fruits.insert(2, "watermelon")

print(fruits)
```

خروجی

```
apple

orange

['banana', 'cherry']

['apple', 'banana', 'kiwi', 'orange']

['apple', 'grape', 'melon', 'orange']

['apple', 'grape', 'watermelon', 'melon', 'orange']
```

حذف اعضای لیست(Python - Remove List Items)

پس از ایجاد یک List، ممکن است بخواهیم یک یا چند عضو آن را حذف کنیم. پایتون روش‌های مختلفی برای حذف اعضای لیست در اختیار ما قرار می‌دهد که هر کدام کاربرد خاص خود را دارند

مهم‌ترین روش‌های حذف اعضای لیست عبارت‌اند از:

- remove()
- pop()
- del

- clear()

حذف عضو بر اساس مقدار (remove) :

اگر مقدار یک عضو را بدانیم و بخواهیم آن را حذف کنیم، از متد remove()استفاده می‌کنیم

ساختار

list.remove(value)

مثال

```
fruits = ["apple", "banana", "cherry"]
fruits.remove("banana")
print(fruits)
```

خروجی

```
['apple', 'cherry']
```

در این مثال، مقدار bananaاز لیست حذف شده است

اگر مقدار تکراری باشد :

اگر چند عضو دارای مقدار یکسان باشند، متد remove()فقط اولین مورد را حذف می‌کند

مثال

```
thislist = ["apple", "banana", "cherry", "banana", "kiwi"]
thislist.remove("banana")
print(thislist)
```

خروجی

```
['apple', 'cherry', 'banana', 'kiwi']
```

مشاهده می‌کنید که فقط اولین bananaحذف شده است

اگر مقدار وجود نداشته باشد

اگر مقدار مورد نظر در لیست وجود نداشته باشد، پایتون خطای ValueErrorتولید می‌کند

مثال

```
fruits = ["apple", "banana"]
fruits.remove("orange")
```

خروجی

```
ValueError: list.remove(x): x not in list
```

حذف عضو بر اساس اندیس(pop) :

اگر بخواهیم عضوی را با استفاده از اندیس (Index) حذف کنیم، از متد pop()استفاده می‌کنیم

ساختار

```
list.pop(index)
```

مثال

```
fruits = ["apple", "banana", "cherry"]
fruits.pop(1)
print(fruits)
```

خروجی

```
['apple', 'cherry']
```

در این مثال، عضو با اندیس 1یعنی bananaحذف شده است

حذف آخرین عضو :

اگر داخل pop()هیچ اندیسی ننویسیم، آخرین عضو لیست حذف می‌شود

مثال

```
fruits = ["apple", "banana", "cherry"]
fruits.pop()
print(fruits)
```

خروجی

```
['apple', 'banana']
```

مقدار حذف‌شده توسط pop() :

یکی از مزایای pop()این است که مقدار حذف‌شده را برمی‌گرداند

مثال

```
fruits = ["apple", "banana", "cherry"]
item = fruits.pop()
print(item)
print(fruits)
```

خروجی

```
cherry
['apple', 'banana']
```

حذف عضو با del :

کلمه کلیدی delنیز می‌تواند برای حذف اعضای لیست استفاده شود

ساختار

```
del list[index]
```

مثال

```
fruits = ["apple", "banana", "cherry"]
del fruits[0]
print(fruits)
```

خروجی

```
['banana', 'cherry']
```

حذف یک بازه از اعضا :

با استفاده از del می‌توان چند عضو را هم‌زمان حذف کرد

مثال

```
numbers = [10, 20, 30, 40, 50]
del numbers[1:4]
print(numbers)
```

خروجی

```
[10, 50]
```

در این مثال، اعضای با اندیس‌های ۱ تا ۳ حذف شده‌اند

حذف کامل لیست :

کلمه del می‌تواند کل لیست را نیز حذف کند

مثال

```
fruits = ["apple", "banana", "cherry"]
del fruits
```

اگر بعد از حذف بنویسیم:

```
print(fruits)
```

خروجی

```
NameError: name 'fruits' is not defined
```

زیرا خود متغیر نیز از حافظه حذف شده است

خالی کردن لیست (clear) :

اگر بخواهیم فقط تمام اعضای لیست حذف شوند اما خود لیست باقی بماند، از متد clear()استفاده می‌کنیم

ساختار

```
list.clear()
```

مثال

```
fruits = ["apple", "banana", "cherry"]
fruits.clear()
print(fruits)
```

خروجی

```
[]
```

در این مثال:

- خود لیست هنوز وجود دارد
- فقط اعضای آن حذف شده‌اند

تفاوت clear و del :

استفاده از()clear

```
numbers = [1,2,3]
numbers.clear()
print(numbers)
```

خروجی

```
[]
```

متغیر هنوز وجود دارد

استفاده از del :

```
numbers = [1,2,3]
del numbers
print(numbers)
```

خروجی

NameError

متغیر کاملاً حذف شده است

مقایسه روش‌های حذف

روش	حذف بر اساس	لیست باقی می‌ماند؟	مقدار حذف‌شده را برمی‌گرداند؟
remove()	مقدار	بله	خیر
pop(index)	اندیس	بله	بله
del list[index]	اندیس	بله	خیر
del list	کل لیست	خیر	خیر
clear()	تمام اعضا	بله	خیر

مثال جامع

```
fruits = ["apple", "banana", "cherry", "kiwi"]
print(fruits)
# حذف بر اساس مقدار
fruits.remove("banana")
print(fruits)
# حذف بر اساس اندیس
fruits.pop(1)
print(fruits)
# اضافه کردن دوباره اعضا
fruits.append("orange")
fruits.append("mango")
print(fruits)
# حذف اولین عضو
del fruits[0]
print(fruits)
# خالی کردن لیست
fruits.clear()
print(fruits)
```

خروجی

```
['apple', 'banana', 'cherry', 'kiwi']
['apple', 'cherry', 'kiwi']
['apple', 'kiwi']
['apple', 'kiwi', 'orange', 'mango']
['kiwi', 'orange', 'mango']
```

[]

چه زمانی از هر روش استفاده کنیم؟

از این روش استفاده کنید	اگر بخواهید...
remove()	حذف یک مقدار مشخص
pop()	حذف یک عضو با اندیس
pop()	حذف و دریافت مقدار حذف‌شده
del	حذف چند عضو با اندیس
del list	حذف کامل متغیر لیست
clear()	خالی کردن لیست و حفظ خود لیست

خطاهای رایج :

1.حذف مقداری که وجود ندارد

```
numbers = [1,2,3]
numbers.remove(5)
```

خروجی

ValueError

2.استفاده از اندیس نامعتبر

```
numbers = [1,2]
numbers.pop(5)
```

خروجی

IndexError

3.استفاده از لیست پس از حذف باdel

```
numbers = [1,2]
del numbers
print(numbers)
```

خروجی

NameError

نکات مهم

- remove()عضو را بر اساس مقدار حذف می‌کند
- اگر مقدار تکراری باشد، فقط اولین مورد حذف می‌شود
- pop()عضو را بر اساس اندیس حذف می‌کند و مقدار حذف‌شده را برمی‌گرداند
- اگر pop()بدون اندیس فراخوانی شود، آخرین عضو حذف می‌شود
- delمی‌تواند یک عضو، یک بازه یا کل لیست را حذف کند
- clear()فقط اعضای لیست را حذف می‌کند و خود لیست همچنان وجود دارد

پیمایش لیست‌ها (Python - Loop Lists) :

یکی از رایج‌ترین کارهایی که در برنامه‌نویسی انجام می‌دهیم، (Looping Iteration)روی اعضای یک لیست است. به عنوان مثال، ممکن است بخواهیم:

- نام تمام دانشجویان را چاپ کنیم
- مجموع نمرات را محاسبه کنیم
- اعضای یک لیست را جستجو کنیم
- روی هر عضو عملی انجام دهیم

در پایتون روش‌های مختلفی برای پیمایش لیست‌ها وجود دارد که مهم‌ترین آن‌ها عبارت‌اند از:

۱. استفاده از حلقه for

۲. استفاده از اندیس‌ها (range() و (len()

۳. استفاده از حلقه while

٤. استفاده از List Comprehension

پیمایش لیست با حلقه for :

ساده‌ترین و متداول‌ترین روش برای پیمایش یک لیست، استفاده از حلقه forاست

در این روش، پایتون به صورت خودکار اعضای لیست را یکی‌یکی در اختیار ما قرار می‌دهد

ساختار کلی

for variable in list_name:

دستورها

مثال

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
  print(x)
```

خروجی

apple

banana

cherry

در این مثال:

- متغیر xدر هر تکرار، یکی از اعضای لیست را دریافت می‌کند
- ابتدا apple
- سپس banana

- سپس cherry

مثال دیگر

```
numbers = [10, 20, 30, 40]
for num in numbers:
   print(num)
```

خروجی

```
10
20
30
40
```

اجرای عملی روی اعضای لیست

در حلقه for می‌توان روی هر عضو عملیات انجام داد.

مثال

```
numbers = [2,4,6,8]
for num in numbers:
   print(num * 2)
```

خروجی

```
4
8
12
16
```

پیمایش با استفاده از اندیس‌ها(Loop Through the Index Numbers)

گاهی لازم است علاوه بر مقدار عضو، شماره اندیس (Index) آن را نیز داشته باشیم

برای این کار معمولاً از توابع ()rangeو ()lenاستفاده می‌کنیم

تابع ()len

تابع ()lenتعداد اعضای لیست را برمی‌گرداند

```
fruits = ["apple", "banana", "cherry"]
print(len(fruits))
```

خروجی

3

تابع ()range

تابع ()rangeدنباله‌ای از اعداد تولید می‌کند

مثلاً

range(5)

اعداد زیر را تولید می‌کند:

0 1 2 3 4

استفاده از range() و len()

```
fruits = ["apple", "banana", "cherry"]
for i in range(len(fruits)):
   print(i, fruits[i])
```

خروجی

```
0 apple
1 banana
2 cherry
```

در این مثال:

- len(fruits) برابر ۳ است
- range(3) اعداد ۰، ۱ و ۲ را تولید می‌کند
- با استفاده از fruits[i] به هر عضو دسترسی پیدا می‌کنیم

مثال دیگر

```
students = ["Ali", "Sara", "Reza"]
for i in range(len(students)):
   print("Student", i + 1, ":", students[i])
```

خروجی

```
Student 1 : Ali
Student 2 : Sara
Student 3 : Reza
```

پیمایش لیست با حلقه while :

روش دیگر برای پیمایش لیست، استفاده از حلقه while است

در این روش باید خودمان اندیس را مدیریت کنیم

مراحل

۱. ایجاد یک متغیر اندیس (i)

۲. شروع از صفر

۳. شرط حلقه

٤. افزایش اندیس

مثال

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
  print(thislist[i])
  i = i + 1
```

خروجی

```
apple
banana
cherry
```

نحوه اجرای حلقه

ابتدا

i = 0

چاپ می‌شود:

apple

سپس

i = 1

چاپ می‌شود:

banana

سپس

i = 2

چاپ می‌شود:

cherry

در پایان

i = 3

شرط حلقه نادرست می‌شود و حلقه پایان می‌یابد

فراموش نکنید اندیس را افزایش دهید!

اگر این دستور را ننویسید:

i = i + 1

حلقه هرگز متوقف نمی‌شود و برنامه وارد حلقه بی‌نهایت (Infinite Loop) خواهد شد

نمونه اشتباه:

```
i = 0
while i < 5:
    print(i)
```

خروجی

0

0

0

...

پیمایش با List Comprehension :

یکی از ویژگی‌های جالب پایتون، List Comprehensionاست

این قابلیت معمولاً برای ساختن لیست‌های جدید استفاده می‌شود، اما می‌توان از آن برای اجرای یک دستور روی تمام اعضای لیست نیز استفاده کرد

مثال

```
thislist = ["apple", "banana", "cherry"]
[print(x) for x in thislist]
```

خروجی

apple

banana

cherry

این دستور معادل کدام کد است؟

```
for x in thislist:
print(x)
```

آیا استفاده از List Comprehension برای چاپ توصیه می‌شود؟

از نظر فنی این کار امکان‌پذیر است، اما بهترین روش برای چاپ اعضای لیست استفاده از حلقه forاست

List Comprehensionبیشتر برای ایجاد لیست‌های جدید طراحی شده است، نه برای چاپ خروجی

مقایسه روش‌های مختلف پیمایش

روش اول (پیشنهادی)

```
for item in fruits:
print(item)
```

مزایا

- ساده
- خوانا
- سریع
- پرکاربرد

روش دوم

```
for i in range(len(fruits)):
print(fruits[i])
```

کاربرد

زمانی که به اندیس نیاز داریم

روش سوم

```
i = 0
while i < len(fruits):
print(fruits[i])
i += 1
```

کاربرد

زمانی که کنترل کامل روی اندیس لازم است

روش چهارم

[print(x) for x in fruits]

کاربرد

کوتاه‌نویسی؛ بیشتر برای تمرین یا مثال‌های ساده مناسب است

مثال کامل

```
fruits = ["apple", "banana", "cherry", "orange"]
print("Using for")
for fruit in fruits:
   print(fruit)
print("----------------")
print("Using index")
for i in range(len(fruits)):
   print(i, fruits[i])
print("----------------")
print("Using while")
i = 0
while i < len(fruits):
   print(fruits[i])
   i += 1
```

خروجی

```
Using for
apple
banana
cherry
orange
```

```
----------
Using index
0 apple
1 banana
2 cherry
3 orange
----------
Using while
apple
banana
cherry
orange
```

چه زمانی از هر روش استفاده کنیم؟

روش	کاربرد
for item in list	بهترین و رایج‌ترین روش برای پیمایش اعضای لیست
for i in range(len(list))	زمانی که به اندیس اعضا نیاز داریم
while	زمانی که کنترل بیشتری روی حلقه لازم است
List Comprehension	برای ساخت لیست‌های جدید یا کدنویسی کوتاه

نکات مهم

- حلقه forساده‌ترین و پرکاربردترین روش پیمایش لیست است
- اگر به شماره اندیس نیاز دارید، از ()rangeو ()lenاستفاده کنید

- در حلقه whileحتماً مقدار اندیس را افزایش دهید تا از ایجاد حلقه بی‌نهایت جلوگیری شود
- اگرچه می‌توان از **List Comprehension**برای چاپ اعضا استفاده کرد، اما هدف اصلی آن ساختن لیست‌های جدید است

List Comprehensionدر پایتون

یکی از ویژگی‌های قدرتمند و محبوب زبان پایتون، List Comprehensionاست

List Comprehensionروشی کوتاه، خوانا و سریع برای ساختن یک لیست جدید (New List) بر اساس یک لیست یا هر شیء قابل پیمایش (Iterable) است , به جای نوشتن چندین خط کد با استفاده از حلقه forو متد ()append، می‌توان همان کار را تنها در یک خط انجام داد

چرا از List Comprehension استفاده می‌کنیم؟

فرض کنید می‌خواهیم از یک لیست میوه‌ها، فقط میوه‌هایی را انتخاب کنیم که حرف aدر نام آن‌ها وجود دارد

بدون استفاده از List Comprehension باید به صورت زیر برنامه بنویسیم

روش معمول) بدون(List Comprehension

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []
for x in fruits:
  if "a" in x:
    newlist.append(x)
print(newlist)
```

خروجی

```
['apple', 'banana', 'mango']
```

در این برنامه:

- ابتدا یک لیست خالی ساخته‌ایم
- سپس روی تمام اعضای لیست پیمایش کرده‌ایم

- شرط را بررسی کرده‌ایم
- در نهایت عضو موردنظر را با append()اضافه کرده‌ایم

همان برنامه باList Comprehension

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]

newlist = [x for x in fruits if "a" in x]

print(newlist)
```

خروجی

```
['apple', 'banana', 'mango']
```

مشاهده می‌کنید که همان نتیجه تنها در یک خط به دست آمده است

مزایایList Comprehension

- کدنویسی کوتاه‌تر
- خوانایی بیشتر
- سرعت اجرای بهتر در بسیاری از موارد
- کاهش احتمال خطا
- مناسب برای ساخت لیست‌های جدید

ساختار(Syntax)

ساختار کلی List Comprehension به صورت زیر است:

```
new_list = [expression for item in iterable if condition]
```

یا به صورت کلی‌تر:

```
newlist = [expression for item in iterable if condition == True]
```

در این ساختار:

- **expression** : مقداری که وارد لیست جدید می‌شود
- **item** : عضو فعلی در حلقه
- **iterable** : شیء قابل پیمایش (مانند List ،Tuple ،Set و ...)
- **condition** : شرط (اختیاری)

اجزایList Comprehension

فرض کنید برنامه زیر را داریم:

newlist = [x for x in fruits if "a" in x]

بخش	توضیح
x	مقداری که وارد لیست جدید می‌شود.
for x in fruits	پیمایش اعضای لیست
if "a" in x	شرط انتخاب اعضا

مقدار بازگشتی

List Comprehensionهمیشه یک لیست جدید تولید می‌کند

لیست اصلی تغییری نمی‌کند

مثال

```
fruits = ["apple", "banana", "kiwi"]
newlist = [x for x in fruits]
print(fruits)
print(newlist)
```

خروجی

```
['apple', 'banana', 'kiwi']
['apple', 'banana', 'kiwi']
```

شرط(Condition)

قسمت شرط مانند یک فیلتر (Filter) عمل می‌کند

فقط اعضایی وارد لیست جدید می‌شوند که شرط برای آن‌ها Trueباشد

مثال

همه میوه‌ها به جزapple

```
fruits = ["apple", "banana", "cherry", "kiwi"]
newlist = [x for x in fruits if x != "apple"]
print(newlist)
```

خروجی

```
['banana', 'cherry', 'kiwi']
```

در این مثال:

if x != "apple"

برای تمام اعضا به جز **apple**مقدار True برمی‌گرداند

شرط اختیاری است

استفاده از ifالزامی نیست

اگر شرطی وجود نداشته باشد، تمام اعضا وارد لیست جدید می‌شوند

```
fruits = ["apple", "banana", "cherry"]
newlist = [x for x in fruits]
print(newlist)
```

خروجی

```
['apple', 'banana', 'cherry']
```

Iterable

قسمت Iterable می‌تواند هر شیء قابل پیمایش باشد

مانند:

- List
- Tuple
- Set
- Dictionary
- String
- range()

استفاده از range()

```
numbers = [x for x in range(10)]
print(numbers)
```

خروجی

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

استفاده از شرط همراه range()

```
numbers = [x for x in range(10) if x < 5]
print(numbers)
```

خروجی

```
[0, 1, 2, 3, 4]
```

در این مثال فقط اعداد کوچک‌تر از ۵ انتخاب شده‌اند

Expression

قسمت **Expression**مقداری است که در لیست جدید قرار می‌گیرد

این مقدار می‌تواند:

- خود عضو
- نتیجه یک عملیات
- خروجی یک تابع
- یا حتی یک شرط باشد

مثال: تبدیل به حروف بزرگ

```
fruits = ["apple", "banana", "kiwi"]
newlist = [x.upper() for x in fruits]
print(newlist)
```

خروجی

```
['APPLE', 'BANANA', 'KIWI']
```

در اینجا قبل از قرار گرفتن عضو در لیست جدید، متد upper()روی آن اجرا شده است

مثال: تبدیل به حروف کوچک

```
words = ["HELLO", "WORLD"]
newlist = [x.lower() for x in words]
print(newlist)
```

خروجی

```
['hello', 'world']
```

قرار دادن مقدار ثابت

Expressionمی‌تواند هر مقداری باشد

مثال

```
fruits = ["apple", "banana", "kiwi"]
newlist = ["hello" for x in fruits]
print(newlist)
```

خروجی

```
['hello', 'hello', 'hello']
```

تعداد اعضای لیست جدید برابر تعداد اعضای لیست اصلی است

استفاده از شرط داخلExpression

گاهی شرط برای فیلتر کردن اعضا نیست، بلکه برای تغییر مقدار خروجی استفاده می‌شود

ساختار

```
value_if_true if condition else value_if_false
```

مثال

```
fruits = ["apple", "banana", "cherry"]
newlist = [x if x != "banana" else "orange" for x in fruits]
print(newlist)
```

خروجی

```
['apple', 'orange', 'cherry']
```

در این مثال:

اگر عضو برابر bananaباشد

به جای آن orangeقرار می‌گیرد

این عبارت چگونه خوانده می‌شود؟

x if x != "banana" else "orange"

یعنی:

اگر عضو bananaنبود، همان عضو را برگردان؛ در غیر این صورت، مقدار orangeرا قرار بده

مثال‌های بیشتر

مربع اعداد

```
numbers = [1,2,3,4,5]
squares = [x*x for x in numbers]
print(squares)
```

خروجی

[1, 4, 9, 16, 25]

اعداد زوج

```
numbers = [1,2,3,4,5,6]
evens = [x for x in numbers if x % 2 == 0]
print(evens)
```

خروجی

[2, 4, 6]

طول رشته‌ها

```
words = ["Ali","Python","Programming"]
lengths = [len(x) for x in words]
print(lengths)
```

خروجی

```
[3, 6, 11]
```

تبدیل رشته‌ها به حروف بزرگ

```
names = ["ali","sara","maryam"]
result = [name.capitalize() for name in names]
print(result)
```

خروجی

```
['Ali', 'Sara', 'Maryam']
```

مقایسه روش معمول و List Comprehension

روش معمول

```
numbers = []
for x in range(10):
    if x % 2 == 0:
        numbers.append(x)
```

روش List Comprehension

```
numbers = [x for x in range(10) if x % 2 == 0]
```

هر دو برنامه دقیقاً یک نتیجه تولید می‌کنند

چه زمانی از List Comprehension استفاده کنیم؟

از List Comprehension زمانی استفاده کنید که:

- بخواهید یک لیست جدید بسازید
- روی تمام اعضای یک لیست عملی انجام دهید
- بخواهید اعضای خاصی را فیلتر کنید
- بخواهید کد کوتاه‌تر و خواناتر باشد

اگر منطق برنامه پیچیده باشد یا چندین شرط و حلقه تو در تو داشته باشید، استفاده از حلقه forمعمولی خواناتر خواهد بود

بخش	توضیح	مثال
Expression	مقداری که وارد لیست جدید می‌شود	x.upper()
for	پیمایش اعضا	for x in fruits
Iterable	منبع داده	fruits، range(10)
if	فیلتر کردن اعضا (اختیاری)	if x != "apple"
شرط درExpression	تغییر مقدار خروجی	x if x != "banana" else "orange"

نکات مهم

- List Comprehensionهمیشه یک لیست جدید ایجاد می‌کند و لیست اصلی را تغییر نمی‌دهد
- بخش ifاختیاری است و فقط برای فیلتر کردن اعضا به کار می‌رود
- بخش Expressionمی‌تواند شامل عملیات، توابع یا حتی یک عبارت شرطی باشد
- هر شیء قابل پیمایش (Iterable) مانند list، tuple، set، stringو ()rangeرا می‌توان در List Comprehensionاستفاده کرد
- از List Comprehension برای ساخت لیست‌های جدید و انجام عملیات ساده و خوانا استفاده کنید، اما برای منطق‌های پیچیده، حلقه forمعمولی انتخاب مناسب‌تری است

مرتب‌سازی لیست‌ها(Python - Sort Lists)

در بسیاری از برنامه‌ها لازم است اعضای یک لیست را مرتب کنیم. برای مثال:

- مرتب کردن اسامی دانشجویان بر اساس حروف الفبا
- مرتب کردن نمرات از کوچک به بزرگ
- مرتب کردن قیمت کالاها از گران‌ترین به ارزان‌ترین
- مرتب کردن داده‌ها قبل از جستجو یا نمایش

پایتون برای مرتب‌سازی لیست‌ها، متد بسیار قدرتمند ()sortرا در اختیار ما قرار می‌دهد

همچنین با استفاده از متد ()reverseمی‌توان ترتیب اعضای لیست را معکوس کرد

در این درس با موارد زیر آشنا می‌شویم:

- مرتب‌سازی الفبایی
- مرتب‌سازی عددی
- مرتب‌سازی نزولی
- مرتب‌سازی سفارشی با key
- مرتب‌سازی بدون حساسیت به حروف بزرگ و کوچک
- معکوس کردن ترتیب لیست با ()reverse

متد ()sort

متد **()sort**اعضای لیست را **در همان لیست (In-place)** مرتب می‌کند

نکته :متد ()sortیک لیست جدید ایجاد نمی‌کند، بلکه لیست اصلی را تغییر می‌دهد

ساختار کلی:

list.sort()

مرتب‌سازی الفبایی(Sort List Alphabetically)

اگر لیست شامل رشته‌ها (Strings) باشد، متد sort()آن‌ها را به صورت الفبایی مرتب می‌کند

مثال

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist)
```

خروجی

```
['banana', 'kiwi', 'mango', 'orange', 'pineapple']
```

در این مثال، رشته‌ها به ترتیب حروف الفبا مرتب شده‌اند

مرتب‌سازی عددی(Sort Numerically)

اگر لیست شامل اعداد باشد، sort()آن‌ها را از کوچک به بزرگ مرتب می‌کند

مثال

```
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

خروجی

```
[23, 50, 65, 82, 100]
```

مرتب‌سازی نزولی(Sort Descending)

به طور پیش‌فرض، متد sort()لیست را به صورت صعودی (Ascending) مرتب می‌کند

برای مرتب‌سازی نزولی (Descending) از پارامتر reverse=Trueاستفاده می‌کنیم

ساختار

```
list.sort(reverse=True)
```

مثال روی رشته‌ها

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse=True)
print(thislist)
```

خروجی

```
['pineapple', 'orange', 'mango', 'kiwi', 'banana']
```

مثال روی اعداد

```
numbers = [100, 50, 65, 82, 23]
numbers.sort(reverse=True)
print(numbers)
```

خروجی

```
[100, 82, 65, 50, 23]
```

مرتب‌سازی سفارشی(Customize Sort Function)

گاهی لازم است لیست بر اساس معیار خاصی مرتب شود

در این حالت از پارامتر **key** استفاده می‌کنیم

ساختار کلی:

```
list.sort(key=function_name)
```

تابعی که به key می‌دهیم، برای هر عضو یک مقدار محاسبه می‌کند و مرتب‌سازی بر اساس آن مقدار انجام می‌شود

مثال: مرتب‌سازی بر اساس فاصله از عدد ۵۰

```
def myfunc(n):
  return abs(n - 50)
thislist = [100, 50, 65, 82, 23]
thislist.sort(key=myfunc)
print(thislist)
```

خروجی

```
[50, 65, 23, 82, 100]
```

توضیح

تابع abs()مقدار قدرمطلق را برمی‌گرداند

فاصله هر عدد از ۵۰:

عدد	فاصله از ۵۰
50	0
65	15
23	27
82	32
100	50

بنابراین مرتب‌سازی بر اساس این فاصله انجام می‌شود

مرتب‌سازی حساس به حروف بزرگ و کوچک(Case Sensitive Sort)

به طور پیش‌فرض، متد sort()به حروف بزرگ و کوچک حساس است.

در جدول کدهای ASCII وUnicode ، حروف بزرگ A) تا (Z قبل از حروف کوچک a) تا (z قرار دارند؛ بنابراین نتیجه ممکن است برای کاربران غیرمنتظره باشد

مثال

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort()
print(thislist)
```

خروجی

```
['Kiwi', 'Orange', 'banana', 'cherry']
```

در این مثال، Kiwiو Orangeقبل از bananaقرار گرفته‌اند، زیرا با حروف بزرگ شروع می‌شوند

مرتب‌سازی بدون حساسیت به حروف(Case Insensitive Sort)

برای اینکه حروف بزرگ و کوچک تفاوتی نداشته باشند، از پارامتر key=str.lowerاستفاده می‌کنیم

مثال

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort(key=str.lower)
print(thislist)
```

خروجی

```
['banana', 'cherry', 'Kiwi', 'Orange']
```

در این حالت، ابتدا همه رشته‌ها برای مقایسه به حروف کوچک تبدیل می‌شوند و سپس مرتب‌سازی انجام می‌شود

تفاوت sort()و reverse()

بسیاری از دانشجویان این دو متد را با هم اشتباه می‌گیرند

sort()

اعضا را **مرتب** می‌کند

reverse()

فقط ترتیب فعلی اعضا را برعکس می‌کند و هیچ مرتب‌سازی انجام نمی‌دهد

معکوس کردن ترتیب لیست(Reverse Order)

متد **()reverse**ترتیب فعلی اعضای لیست را معکوس می‌کند

ساختار:

```
list.reverse()
```

مثال

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```

خروجی

```
['cherry', 'Kiwi', 'Orange', 'banana']
```

در این مثال، هیچ مرتب‌سازی انجام نشده است؛ فقط ترتیب اعضا برعکس شده است

مقایسه sort(reverse=True)و ()reverse

فرض کنید لیست زیر را داریم:

```
numbers = [50, 10, 80, 30]
```

sort(reverse=True)استفاده از

```
numbers.sort(reverse=True)
print(numbers)
```

خروجی

```
[80, 50, 30, 10]
```

ابتدا لیست مرتب شده و سپس به صورت نزولی نمایش داده شده است

استفاده از reverse()

```
numbers = [50, 10, 80, 30]
numbers.reverse()
print(numbers)
```

خروجی

```
[30, 80, 10, 50]
```

در اینجا فقط ترتیب اعضا برعکس شده و مرتب‌سازی انجام نشده است

مثال کامل

```
fruits = ["orange", "mango", "kiwi", "pineapple", "banana"]
print("Original List:")
print(fruits)
fruits.sort()
print("\nAscending:")
print(fruits)
fruits.sort(reverse=True)
print("\nDescending:")
print(fruits)
fruits.reverse()
print("\nReverse Order:")
print(fruits)
```

خروجی

```
Original List:
['orange', 'mango', 'kiwi', 'pineapple', 'banana']
```

Ascending:

['banana', 'kiwi', 'mango', 'orange', 'pineapple']

Descending:

['pineapple', 'orange', 'mango', 'kiwi', 'banana']

Reverse Order:

['banana', 'kiwi', 'mango', 'orange', 'pineapple']

جدول خلاصه

متد	کاربرد	مثال
sort()	مرتب‌سازی صعودی	list.sort()
sort(reverse=True)	مرتب‌سازی نزولی	list.sort(reverse=True)
sort(key=function)	مرتب‌سازی سفارشی	list.sort(key=myfunc)
sort(key=str.lower)	مرتب‌سازی بدون حساسیت به بزرگی و کوچکی حروف	list.sort(key=str.lower)
reverse()	معکوس کردن ترتیب فعلی لیست	list.reverse()

نکات مهم

- متد sort()لیست را در همان محل (In-place) مرتب می‌کند و مقدار جدیدی برنمی‌گرداند
- مرتب‌سازی پیش‌فرض برای اعداد و رشته‌ها به صورت صعودی است
- با استفاده از reverse=Trueمی‌توان مرتب‌سازی نزولی انجام داد
- پارامتر keyامکان تعریف معیار دلخواه برای مرتب‌سازی را فراهم می‌کند
- مرتب‌سازی پیش‌فرض به بزرگی و کوچکی حروف حساس است؛ برای نادیده گرفتن این تفاوت از key=str.lower استفاده کنید
- متد reverse()فقط ترتیب فعلی اعضا را برعکس می‌کند و هیچ مرتب‌سازی انجام نمی‌دهد

کپی کردن لیست‌ها(Python - Copy Lists)

در بسیاری از برنامه‌ها لازم است از یک لیست، یک نسخه مستقل (Copy) تهیه کنیم تا بتوانیم بدون تغییر دادن لیست اصلی، روی نسخه جدید عملیات انجام دهیم

یکی از اشتباهات رایج برنامه‌نویسان مبتدی این است که تصور می‌کنند با نوشتن:

list2 = list1

یک کپی از لیست ساخته می‌شود اما این تصور اشتباه است در واقع، هر دو متغیر به یک لیست مشترک در حافظه اشاره می‌کنند.

چرا list2 = list1کپی نیست؟

به مثال زیر توجه کنید.

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist
mylist[0] = "orange"
print(thislist)
print(mylist)
```

خروجی

```
['orange', 'banana', 'cherry']
['orange', 'banana', 'cherry']
```

مشاهده می‌کنید که با تغییر mylist، لیست اصلی نیز تغییر کرده است

دلیل آن این است که:

- thislist
- mylist

هر دو به یک شیء (Object) در حافظه اشاره می‌کنند و به این حالت) Referenceارجاع (گفته می‌شود

روش اول: استفاده از copy()

ساده‌ترین روش برای تهیه یک نسخه مستقل، استفاده از متد copy()است

ساختار

```
new_list = old_list.copy()
```

مثال

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)
```

خروجی

```
['apple', 'banana', 'cherry']
```

بررسی استقلال دو لیست

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
mylist[0] = "orange"
print(thislist)
print(mylist)
```

خروجی

```
['apple', 'banana', 'cherry']
['orange', 'banana', 'cherry']
```

در این مثال:

- لیست اصلی تغییر نکرده است
- فقط نسخه کپی شده تغییر کرده است

تصویر مفهومی

thislist ———► ["apple","banana","cherry"]

mylist ————► ["apple","banana","cherry"]

این دو لیست اکنون **مستقل** هستند

روش دوم: استفاده از list()

روش دیگر، استفاده از سازنده **list()**است

ساختار

new_list = list(old_list)

مثال

```
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

خروجی

['apple', 'banana', 'cherry']

بررسی

```
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
mylist.append("orange")
print(thislist)
print(mylist)
```

خروجی

['apple', 'banana', 'cherry']

['apple', 'banana', 'cherry', 'orange']

روش سوم: استفاده از عملگرSlice

روش بسیار رایج دیگر، استفاده از عملگر **(:) Slice**است

ساختار

new_list = old_list[:]

مثال

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist[:]
print(mylist)
```

خروجی

['apple', 'banana', 'cherry']

بررسی

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist[:]
mylist.remove("banana")
print(thislist)
print(mylist)
```

خروجی

['apple', 'banana', 'cherry']

['apple', 'cherry'] مشاهده می‌کنید که لیست اصلی تغییری نکرده است

مقایسه سه روش

روش اول

copy1 = mylist.copy()

روش دوم

copy2 = list(mylist)

روش سوم

copy3 = mylist[:] هر سه روش برای لیست‌های معمولی (سطحی) نتیجه یکسانی دارند

مثال کامل

```
fruits = ["apple", "banana", "cherry"]
copy1 = fruits.copy()
copy2 = list(fruits)
copy3 = fruits[:]
copy1.append("orange")
copy2.remove("banana")
copy3[0] = "kiwi"
print("Original :", fruits)
print("Copy 1 :", copy1)
print("Copy 2 :", copy2)
print("Copy 3 :", copy3)
```

خروجی

```
Original : ['apple', 'banana', 'cherry']
Copy 1 : ['apple', 'banana', 'cherry', 'orange']
Copy 2 : ['apple', 'cherry']
Copy 3 : ['kiwi', 'banana', 'cherry']
```

مشاهده می‌کنید که هیچ تغییری در لیست اصلی ایجاد نشده است

تفاوت Reference وCopy

Reference

list2 = list1

list1 ──┐

│

▼

[10,20,30]

▲

│

list2 ──┘

هر دو متغیر به یک لیست اشاره می‌کنند

Copy

list2 = list1.copy()

list1 ──► [10,20,30]

list2 ──► [10,20,30]

دو لیست کاملاً مستقل هستند

نکته مهم: کپی سطحی(Shallow Copy)

سه روش معرفی‌شده(copy()) ، ()listو [:]فقط یک کپی سطحی (Shallow Copy) ایجاد می‌کنند

اگر لیست شامل عناصر ساده مانند اعداد، رشته‌ها یا مقادیر بولی باشد، این روش‌ها کاملاً کافی هستند

اما اگر لیست شامل لیست‌های تو در تو (Nested Lists) یا اشیای قابل تغییر دیگر باشد، عناصر داخلی همچنان بین دو لیست مشترک خواهند بود

مثال

```
numbers = [[1, 2], [3, 4]]

copy_numbers = numbers.copy()

copy_numbers[0][0] = 100

print(numbers)

print(copy_numbers)
```

خروجی

```
[[100, 2], [3, 4]]

[[100, 2], [3, 4]]
```

دلیل این رفتار آن است که فقط لیست اصلی کپی شده است، اما لیست‌های داخلی همچنان مشترک هستند

برای ساخت یک کپی عمیق (Deep Copy) از ساختارهای تو در تو، از تابع deepcopy()در ماژول copyاستفاده می‌شود که معمولاً در مباحث پیشرفته‌تر آموزش داده می‌شود

جدول مقایسه روش‌های کپی

روش	نحوه استفاده	لیست جدید ایجاد می‌شود؟	کاربرد
انتساب(=)	list2 = list1	خیر	فقط ایجاد ارجاع(Reference)
copy()	list2 = list1.copy()	بله	بهترین و خواناترین روش
list()	list2 = list(list1)	بله	استفاده از سازنده لیست
Slice	list2 = list1[:]	بله	روش کوتاه و رایج

نکات مهم

- عبارت list2 = list1کپی ایجاد نمی‌کند؛ فقط یک ارجاع جدید به همان لیست می‌سازد
- برای ایجاد یک نسخه مستقل از لیست می‌توان از copy()، list()یا عملگر Slice ([:]) استفاده کرد
- هر سه روش معرفی‌شده کپی سطحی (Shallow Copy) ایجاد می‌کنند
- برای لیست‌های ساده، این روش‌ها کاملاً مناسب هستند

- برای لیست‌های تو در تو، در صورت نیاز به یک نسخه کاملاً مستقل، باید از **Deep Copy**استفاده شود

ادغام لیست‌ها(Python - Join Lists)

در بسیاری از برنامه‌ها لازم است دو یا چند لیست را با یکدیگر ترکیب Join) یا (Concatenate کنیم

برای مثال:

- ترکیب لیست اسامی دو کلاس
- ادغام لیست نمرات دو امتحان
- ترکیب لیست محصولات دو فروشگاه
- ساخت یک لیست جدید از چند لیست مختلف

پایتون چندین روش برای ادغام لیست‌ها در اختیار ما قرار می‌دهد که مهم‌ترین آن‌ها عبارت‌اند از:

۱. استفاده از عملگر +
۲. استفاده از حلقه forو متد ()append
۳. استفاده از متد ()extend

در ادامه هر یک از این روش‌ها را بررسی می‌کنیم.

روش اول: استفاده از عملگر +

ساده‌ترین روش برای ترکیب دو لیست، استفاده از عملگر +است

این عملگر دو لیست را به هم متصل کرده و **یک لیست جدید** ایجاد می‌کند

ساختار

```
list3 = list1 + list2
```

مثال

```
list1 = ["a", "b", "c"]

list2 = [1, 2, 3]

list3 = list1 + list2

print(list3)
```

خروجی

```
['a', 'b', 'c', 1, 2, 3]
```

در این مثال:

- list1بدون تغییر باقی می‌ماند
- list2بدون تغییر باقی می‌ماند
- نتیجه در list3ذخیره می‌شود

بررسی

```
list1 = ["a", "b"]

list2 = [1, 2]

list3 = list1 + list2

print(list1)

print(list2)

print(list3)
```

خروجی

```
['a', 'b']

[1, 2]

['a', 'b', 1, 2]
```

ادغام بیش از دو لیست

عملگر +می‌تواند چندین لیست را نیز با هم ترکیب کند

```
list1 = [1,2]
list2 = [3,4]
list3 = [5,6]
result = list1 + list2 + list3
print(result)
```

خروجی

```
[1, 2, 3, 4, 5, 6]
```

روش دوم: استفاده از حلقه forو متد ()append

در این روش، اعضای لیست دوم را یکی‌یکی به انتهای لیست اول اضافه می‌کنیم

مثال

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
for x in list2:
  list1.append(x)
print(list1)
```

خروجی

```
['a', 'b', 'c', 1, 2, 3]
```

این حلقه چگونه کار می‌کند؟

ابتدا:

```
x = 1
list1.append(1)
```

لیست می‌شود:

```
['a', 'b', 'c', 1]
```

سپس:

```
x = 2
['a', 'b', 'c', 1, 2]
```

در نهایت:

```
x = 3
['a', 'b', 'c', 1, 2, 3]
```

تفاوت این روش با عملگر +

در این روش:

- خود list1 تغییر می‌کند.
- لیست جدید ساخته نمی‌شود.

مثال

```
list1 = ["a", "b"]
list2 = [1,2]
for x in list2:
   list1.append(x)
print(list1)
print(list2)
```

خروجی

['a', 'b', 1, 2]

[1, 2]

روش سوم: استفاده از ()extend

بهترین و رایج‌ترین روش برای اضافه کردن تمام اعضای یک لیست به لیست دیگر، استفاده از متد ()extendاست

ساختار

list1.extend(list2)

مثال

```
list1 = ["a", "b", "c"]
list2 = [1,2,3]
list1.extend(list2)
print(list1)
```

خروجی

['a', 'b', 'c', 1, 2, 3]

متد ()extendچگونه کار می‌کند؟

متد ()extendتمام اعضای لیست دوم را به انتهای لیست اول اضافه می‌کند

در واقع این دستور تقریباً معادل کد زیر است:

```
for item in list2:
   list1.append(item)
```

بنابراین ()extendهمان کار حلقه forو ()appendرا انجام می‌دهد، اما کوتاه‌تر و خواناتر است

تفاوت append()و extend()

این دو متد را نباید با یکدیگر اشتباه گرفت

استفاده از append()

```
list1 = [1,2]
list2 = [3,4]
list1.append(list2)
print(list1)
```

خروجی

```
[1, 2, [3, 4]]
```

در اینجا کل لیست دوم به عنوان یک عضو به لیست اول اضافه شده است

استفاده از extend()

```
list1 = [1,2]
list2 = [3,4]
list1.extend(list2)
print(list1)
```

خروجی

```
[1, 2, 3, 4]
```

در این حالت، اعضای لیست دوم به صورت جداگانه به لیست اول اضافه شده‌اند

مثال کامل

```
list1 = ["Ali", "Sara"]
list2 = ["Reza", "Maryam"]
print("Original List 1")
print(list1)
print("Original List 2")
print(list2)
list1.extend(list2)
print("Joined List")
print(list1)
```

خروجی

```
Original List 1
['Ali', 'Sara']
Original List 2
['Reza', 'Maryam']
Joined List
['Ali', 'Sara', 'Reza', 'Maryam']
```

ادغام لیست‌های عددی

```
numbers1 = [10,20,30]
numbers2 = [40,50,60]
numbers3 = numbers1 + numbers2
print(numbers3)
```

خروجی

```
[10, 20, 30, 40, 50, 60]
```

ادغام لیست‌های رشته‌ای

```
letters1 = ["A","B","C"]
letters2 = ["D","E","F"]
letters1.extend(letters2)
print(letters1)
```

خروجی

```
['A', 'B', 'C', 'D', 'E', 'F']
```

جدول مقایسه روش‌های ادغام

کاربرد	لیست اصلی تغییر می‌کند؟	لیست جدید ایجاد می‌شود؟	روش
ساخت یک لیست جدید	خیر	بله	+
افزودن تدریجی اعضا	بله	خیر	append() در حلقه for
افزودن تمام اعضای یک لیست به لیست دیگر	بله	خیر	extend()

چه زمانی از هر روش استفاده کنیم؟

روش مناسب	اگر بخواهید...
+	یک لیست جدید از دو لیست بسازید
extend()	اعضای یک لیست را به انتهای لیست دیگر اضافه کنید
for + append()	هنگام اضافه کردن روی هر عضو عملیات انجام دهید

نکات مهم

- عملگر +دو یا چند لیست را ترکیب کرده و یک لیست جدید ایجاد می‌کند
- متد ()extendاعضای یک لیست را به انتهای لیست دیگر اضافه می‌کند و لیست جدیدی نمی‌سازد
- استفاده از ()extendمعمولاً از نوشتن حلقه forو فراخوانی مکرر ()appendساده‌تر و خواناتر است
- متد ()appendاگر یک لیست را دریافت کند، آن را به عنوان یک عضو واحد به لیست اضافه می‌کند؛ در حالی که ()extendاعضای آن لیست را به صورت جداگانه اضافه می‌کند

متدهای لیست در پایتون(Python - List Methods)

در زبان پایتون، Listیکی از پرکاربردترین ساختمان‌های داده است. برای کار با لیست‌ها، پایتون مجموعه‌ای از متدهای داخلی (Built-in Methods)را در اختیار برنامه‌نویسان قرار می‌دهد

این متدها امکان انجام عملیات مختلفی مانند:

- اضافه کردن عضو
- حذف عضو
- جستجو
- شمارش
- مرتب‌سازی
- کپی گرفتن
- معکوس کردن ترتیب اعضا

و بسیاری از عملیات دیگر را فراهم می‌کنند

در این درس با تمام متدهای مهم لیست آشنا می‌شویم

جدول متدهای لیست

متد	توضیح
append()	اضافه کردن یک عضو به انتهای لیست
clear()	حذف تمام اعضای لیست
copy()	ایجاد یک کپی از لیست
count()	شمارش تعداد تکرار یک مقدار
extend()	اضافه کردن اعضای یک لیست یا Iterable دیگر
index()	یافتن اندیس اولین عضو با مقدار مشخص
insert()	درج یک عضو در محل مشخص
pop()	حذف عضو بر اساس اندیس
remove()	حذف عضو بر اساس مقدار
reverse()	معکوس کردن ترتیب اعضا
sort()	مرتب کردن اعضای لیست

append() متد-1

این متد یک عضو را به **انتهای لیست** اضافه می‌کند

ساختار

list.append(value)

مثال

```
fruits = ["apple", "banana"]
fruits.append("orange")
print(fruits)
```

خروجی

```
['apple', 'banana', 'orange']
```

2-متد clear()

تمام اعضای لیست را حذف می‌کند، اما خود لیست باقی می‌ماند

مثال

```
numbers = [10,20,30]
numbers.clear()
print(numbers)
```

خروجی

```
[]
```

3-متد copy()

یک نسخه مستقل (Shallow Copy) از لیست ایجاد می‌کند

مثال

```
list1 = [1,2,3]
list2 = list1.copy()
print(list2)
```

خروجی

```
[1, 2, 3]
```

4-متد count()

تعداد تکرار یک مقدار در لیست را برمی‌گرداند

ساختار

```
list.count(value)
```

مثال

```
numbers = [1,2,2,3,2,4]
print(numbers.count(2))
```

خروجی

3

در این مثال، عدد **2**سه بار در لیست تکرار شده است

5-متد extend()

تمام اعضای یک لیست (یا هر شیء قابل پیمایش) را به انتهای لیست فعلی اضافه می‌کند

مثال

```
list1 = [1,2]
list2 = [3,4]
list1.extend(list2)
print(list1)
```

خروجی

[1, 2, 3, 4]

6-متد index()

اندیس **اولین** عضو با مقدار مشخص را برمی‌گرداند

ساختارlist.index(value)

مثال

```
fruits = ["apple","banana","cherry"]
print(fruits.index("banana"))
```

خروجی

1

اگر مقدار موردنظر در لیست وجود نداشته باشد، خطای ValueErrorایجاد می‌شود

7-متد insert()

یک عضو را در محل مشخص وارد لیست می‌کند

ساختار

list.insert(index, value)

مثال

```
numbers = [10,20,30]
numbers.insert(1,15)
print(numbers)
```

خروجی

```
[10, 15, 20, 30]
```

8-متد pop()

عضو موجود در اندیس مشخص را حذف کرده و مقدار آن را برمی‌گرداند

مثال

```
numbers = [10,20,30]
item = numbers.pop(1)
print(item)
print(numbers)
```

خروجی

```
20
[10, 30]
```

اگر اندیسی مشخص نشود، آخرین عضو حذف می‌شود

9-متد remove()

اولین عضوی که مقدار مشخصی دارد را حذف می‌کند

مثال

```
fruits = ["apple","banana","cherry"]
fruits.remove("banana")
print(fruits)
```

خروجی

```
['apple', 'cherry']
```

اگر مقدار در لیست وجود نداشته باشد، خطای ValueErrorایجاد می‌شود

10-متد reverse()

ترتیب فعلی اعضای لیست را برعکس می‌کند

مثال

```
numbers = [10,20,30,40]
numbers.reverse()
print(numbers)
```

خروجی

```
[40, 30, 20, 10]
```

این متد مرتب‌سازی انجام نمی‌دهد؛ فقط ترتیب فعلی را معکوس می‌کند

11-متد sort()

اعضای لیست را مرتب می‌کند.

مثال

```
numbers = [50,10,30,20]
numbers.sort()
print(numbers)
```

خروجی

```
[10, 20, 30, 50]
```

مرتب‌سازی نزولی

```
numbers = [50,10,30,20]
numbers.sort(reverse=True)
print(numbers)
```

خروجی

```
[50, 30, 20, 10]
```

مثال جامع

```
numbers = [10, 20, 30]
# اضافه کردن عضو
numbers.append(40)
# درج عضو
numbers.insert(1, 15)
# شمارش
print(numbers.count(20))
# یافتن اندیس
print(numbers.index(30))
# حذف عضو
numbers.remove(15)
# حذف آخرین عضو
numbers.pop()
# مرتب‌سازی
numbers.sort()
# معکوس کردن
numbers.reverse()
print(numbers)
```

خروجی

```
1
2
[30, 20, 10]
```

جدول خلاصه متدها

متد	مقدار بازگشتی	تغییر در لیست
append()	ندارد(None)	yes
clear()	ندارد	yes
copy()	لیست جدید	no
count()	تعداد تکرار	no
extend()	ندارد	yes
index()	شماره اندیس	no
insert()	ندارد	yes
pop()	عضو حذف‌شده	yes
remove()	ندارد	yes
reverse()	ندارد	yes
sort()	ندارد	yes

نکات مهم

- بیشتر متدهای لیست مانند append()، sort()، reverse()، insert()و remove()لیست اصلی را تغییر می‌دهند و مقدار بازگشتی آن‌ها Noneاست
- متد copy()تنها متدی است که یک لیست جدید ایجاد می‌کند
- متد count()تعداد تکرار یک مقدار را برمی‌گرداند
- متد index()فقط اولین محل وقوع مقدار را پیدا می‌کند
- متد pop()علاوه بر حذف عضو، مقدار آن را نیز برمی‌گرداند
- متد remove()بر اساس مقدار حذف می‌کند، در حالی که pop()بر اساس اندیس عمل می‌کند
- متد sort()اعضا را مرتب می‌کند، اما reverse()فقط ترتیب فعلی آن‌ها را برعکس می‌کند